

算法竞赛数据结构逐步学习讲义

这份讲义按你的原文档思路展开：不要把数据结构当模板背，而是从题目操作出发，追问：题目反复要我维护什么信息？暴力为什么慢？这个信息有没有合并性、前缀性、有序性、关系性、历史版本性？然后再选择数据结构。

文档正式详细讲的核心数据结构是：邻接表、并查集、树状数组、ST 表、线段树、LCA、Trie、平衡树 / 权值线段树、可持久化线段树。文档里还提到了前缀和、差分、堆、拓扑排序、KMP、哈希、AC 自动机、树链剖分、分块、莫队等，我会放在最后的“扩展地图”里，告诉你它们分别该在什么场景出现。

0. 数据结构到底在优化什么？

竞赛里的数据结构，本质不是“高级模板”，而是“减少重复劳动”。

比如题目有一个数组 `a[1..n]`，不断询问区间和：

```
for (int i = l; i <= r; i++) ans += a[i];
```

这叫每次重新扫描。你没有把历史计算结果组织起来。

前缀和把信息组织成：

```
s[i] = a[1] + a[2] + ... + a[i];
```

于是区间和变成：

```
s[r] - s[l - 1]
```

查询从 $O(n)$ 变成 $O(1)$ 。

但是如果题目又加入修改：

```
修改 a[x]  
查询 [l, r]  
修改 a[y]  
查询 [l, r]
```

普通前缀和就失效了，因为改一个点会影响后面很多前缀和。于是你需要树状数组或线段树。

所以你以后拿到题先问：

1. 数据对象是什么：数组、图、树、字符串、集合、历史版本？

2. 数据会不会改：静态还是动态？
3. 操作对象是点、区间、路径、集合关系，还是前缀？
4. 答案能不能由小块合并成大块？
5. 题目是否只问关系，不问具体路径？
6. 题目是否问历史版本？

这六问就是从“看题”到“想到结构”的桥。

一、邻接表：图和树的底层存储

1. 它的作用是什么？

邻接表用来存图和树。

如果图有 n 个点、 m 条边，邻接矩阵要开：

```
bool g[n][n];
```

空间是 $O(n^2)$ 。

当 $n = 100000$ 时， $n^2 = 10^{10}$ ，内存直接爆炸。

邻接表只存真实存在的边：

```
vector<int> g[N];
```

如果有一条边 $u \rightarrow v$ ：

```
g[u].push_back(v);
```

空间是 $O(n + m)$ 。

这就是邻接表的核心价值：不再存所有可能边，只存真实存在的边。

2. 为什么会有这种结构？

图的数学定义是：

$$G = (V, E)$$

其中：

- V 是点集。

- E 是边集。

邻接矩阵记录的是：

```
exist[u][v] = u 到 v 是否有边
```

它关心所有点对，即使大多数点对没有边。

邻接表记录的是：

```
Adj(u) = 所有从 u 出发能到达的点
```

它只关心真实存在的边。

竞赛中大部分图是稀疏图，也就是 m 远小于 n^2 。所以邻接表几乎是图论默认存法。

3. 怎么实现？

无权图

```
vector<int> g[N];

void add_edge(int u, int v) {
    g[u].push_back(v);
}
```

无向图要加两次：

```
g[u].push_back(v);
g[v].push_back(u);
```

带权图

```
struct Edge {
    int to;
    int w;
};

vector<Edge> g[N];

void add_edge(int u, int v, int w) {
    g[u].push_back({v, w});
}
```

链式前向星

当边数非常大、追求常数时，也可以写链式前向星：

```
struct Edge {
    int to, next, w;
} e[M];

int head[N], cnt;

void add(int u, int v, int w) {
    e[++cnt] = {v, head[u], w};
    head[u] = cnt;
}
```

初学阶段先熟练 `vector` 邻接表即可。

4. 能解决哪类问题？

凡是图或树上要遍历、转移、找路径，第一步几乎都是邻接表。

典型包括：

- DFS / BFS。
- 最短路。
- 树上 DFS。
- LCA。
- 树形 DP。
- 拓扑排序。
- 最小生成树中的建图。

5. 做题时怎么想到？

看到这些信号就想邻接表：

- 题目给 `n` 个点、`m` 条边。
- `n` 很大，比如 `1e5`、`1e6`。
- 需要从某点出发遍历能到哪些点。
- 需要存树。
- 需要存带权边。
- 图显然不是完全图。

一句话：只要是图，先默认邻接表；只有 `n` 很小且需要 $O(1)$ 判断两点是否直接相连时，才考虑邻接矩阵。

6. Luogu 练习路线

入门：P5318 查找文献

题意：给一个有向图，从 1 号点开始分别输出 DFS 和 BFS 顺序。如果能去多个点，要先去编号小的点。

为什么用邻接表？

因为 `n` 和 `m` 都可能很大，用邻接矩阵会爆。并且每个点只需要访问真实出边。

核心代码：

```
vector<int> g[N];
bool vis[N];

void dfs(int u) {
    cout << u << ' ';
    vis[u] = true;
    for (int v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}

void bfs(int s) {
    queue<int> q;
    q.push(s);
    vis[s] = true;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << ' ';
        for (int v : g[u]) {
            if (!vis[v]) {
                vis[v] = true;
                q.push(v);
            }
        }
    }
}
```

逐句理解：

- `vector<int> g[N]`：每个点开一个动态数组，里面存它连出去的点。
- `vis[u]`：防止图中一个点被多条路径重复访问。
- `sort(g[i].begin(), g[i].end())`：因为题目要求优先访问编号小的点。
- DFS 是“一条路先走到底”。
- BFS 是“先发现的点先扩展”，所以用队列。

坑点：

1. 忘记排序，遍历顺序错。
2. DFS 后没有清空 `vis`，导致 BFS 输出不完整。
3. 有向边只加 `g[x].push_back(y)`，不要误加反边。

进阶：P3379 最近公共祖先

这题核心是 LCA，但第一步仍然是邻接表存树。

进阶：P4551 最长异或路径

这题需要存带权树：

```
struct Edge { int to, w; };  
vector<Edge> g[N];
```

然后用 DFS 求根到每个点的异或值，再配合 01-Trie。

二、并查集：维护合并后的集合关系

1. 它的作用是什么？

并查集维护的问题是：

一开始每个元素各自独立。
不断合并两个元素所在集合。
不断询问两个元素是否在同一集合。

它不关心两点之间具体路径，只关心“是否属于同一个集合”。

2. 为什么会有这种结构？

如果每次询问 x 和 y 是否连通，都重新 DFS/BFS，复杂度可能很高。

并查集的想法是：每个集合选一个代表元，称为根。

如果：

```
find(x) == find(y)
```

就说明 x 和 y 在同一集合。

这把“图连通性问题”变成了“根是否相同”。

3. 数学原理：等价关系

并查集维护的是一种等价关系。

等价关系有三个性质：

1. 自反性：x 和 x 同集合。
2. 对称性：如果 x 和 y 同集合，那么 y 和 x 同集合。
3. 传递性：如果 x 和 y 同集合，y 和 z 同集合，那么 x 和 z 同集合。

并查集本质就是高效维护这种传递闭包。

4. 怎么实现？

```
int fa[N], sz[N];

int find(int x) {
    if (fa[x] == x) return x;
    return fa[x] = find(fa[x]);
}

void merge_set(int x, int y) {
    int fx = find(x);
    int fy = find(y);
    if (fx == fy) return;
    if (sz[fx] < sz[fy]) swap(fx, fy);
    fa[fy] = fx;
    sz[fx] += sz[fy];
}
```

逐句理解：

- `fa[x]`：x 的父亲。
- `fa[x] == x`：x 是所在集合的根。
- `find(x)`：沿父亲指针一路向上找根。
- `fa[x] = find(fa[x])`：路径压缩，把 x 直接挂到根上。
- `sz[x]`：只有 x 是根时，表示这个集合大小。
- `sz[fx] < sz[fy]` 时交换：让小集合挂到大集合，树不容易变高。

5. 它优化了什么？

暴力合并和查询可能要扫描很多点。

并查集在路径压缩 + 按大小合并后，单次操作均摊复杂度接近 $O(1)$ ，通常写作 $O(\alpha(n))$ 。

你不用害怕 $\alpha(n)$ ，竞赛中它几乎可以看作常数。

6. 做题时怎么想到？

看到这些关键词就想并查集：

- 合并集合。
- 是否连通。

- 是否同组。
- 朋友关系、家族关系、帮派关系。
- 加边后判断连通性。
- Kruskal 最小生成树。
- 只问“是否连通”，不问“怎么走”。

7. Luogu 练习路线

入门：P3367 并查集

操作：

```
1 x y: 合并 x 和 y 所在集合
2 x y: 询问是否同集合
```

为什么想到并查集？

因为题目没有问路径，只问集合关系。

进阶：P1197 星球大战

难点：删边不好维护连通性。

思维反转：把“删点/删边”离线倒过来，变成“加点/加边”。并查集擅长加边合并，不擅长在线删边。

为什么能倒着做？

正向：星球被摧毁，连通块可能变多。

反向：星球被恢复，连通块只会合并，不会拆开。

这时并查集就能做。

坑点：

1. 初始时要先把所有没有被摧毁的点加入。
2. 倒序恢复点时，只和已经存在的邻点合并。
3. 每次恢复一个点，连通块数先 `+1`，每成功合并一次再 `-1`。

进阶：P1967 货车运输

这题先用 Kruskal 建最大生成树。Kruskal 需要并查集判断两点是否已经连通。

之后在最大生成树上用 LCA 查询路径最小边。

这是一道“并查集 + LCA”的综合题。

三、树状数组：把动态前缀和做成二进制结构

1. 它的作用是什么？

树状数组主要解决：

1. 单点修改 + 区间求和。
2. 区间修改 + 单点查询。
3. 逆序对、动态排名、离线区间统计等前缀可加信息。

2. 为什么会有这种结构？

普通数组：

- 单点修改 $O(1)$ 。
- 区间求和 $O(n)$ 。

前缀和：

- 区间求和 $O(1)$ 。
- 单点修改会影响一大片前缀， $O(n)$ 。

树状数组折中：

- 修改 $O(\log n)$ 。
- 查询 $O(\log n)$ 。

它适合“信息可以做前缀累加”的问题。

3. 数学原理：lowbit

树状数组定义：

```
tree[i] = a[i - lowbit(i) + 1] + ... + a[i]
```

也就是说， $tree[i]$ 管一个以 i 结尾、长度为 $lowbit(i)$ 的区间。

$lowbit(x)$ 是 x 二进制中最低位的 1 所代表的值：

```
int lowbit(int x) {  
    return x & -x;  
}
```

例如：

```
12 = 1100(2)
lowbit(12) = 0100(2) = 4
```

为什么 `x & -x` 能取最低位的 1?

因为计算机中 `-x = ~x + 1`。在二进制中，最低位的那个 1 右边全是 0，取反加一后，只有最低位的 1 能和原数同时保留为 1，更高位会相反，右边仍是 0。

4. 为什么 query 是 `x -= lowbit(x)`?

查询前缀和 `sum(1..x)`：

```
long long query(int x) {
    long long ans = 0;
    while (x > 0) {
        ans += tree[x];
        x -= lowbit(x);
    }
    return ans;
}
```

因为 `tree[x]` 维护：

```
[x - lowbit(x) + 1, x]
```

加完这一段，剩下的前缀末尾变成：

```
x - lowbit(x)
```

于是继续跳。

例：查询 `sum(1..13)`：

```
13 = 1101, tree[13] 管 [13,13]
12 = 1100, tree[12] 管 [9,12]
8  = 1000, tree[8]  管 [1,8]
```

所以：

```
[1,13] = [13,13] + [9,12] + [1,8]
```

不重不漏。

5. 为什么 update 是 `x += lowbit(x)` ?

修改 `a[pos] += k` 后，所有包含 `pos` 的 `tree[i]` 都要加 `k`。

树状数组保证从 `pos` 出发不断：

```
x += lowbit(x)
```

就会跳到所有包含这个位置的上级区间。

这正好和查询相反：

- 查询：往左拆前缀。
- 修改：往右找包含自己的大块。

6. 基础实现

```
long long tree[N];
int n;

int lowbit(int x) {
    return x & -x;
}

void add(int x, long long k) {
    while (x <= n) {
        tree[x] += k;
        x += lowbit(x);
    }
}

long long query(int x) {
    long long ans = 0;
    while (x > 0) {
        ans += tree[x];
        x -= lowbit(x);
    }
    return ans;
}

long long range_sum(int l, int r) {
    return query(r) - query(l - 1);
}
```

7. Luogu 练习路线

入门：P3374 树状数组 1

操作：

```
1 x k: a[x] += k
2 x y: 查询 [x,y] 的和
```

为什么想到树状数组？

因为区间和可以拆成两个前缀和，而题目有单点修改。

坑点：

1. 下标从 1 开始，不能从 0 开始。
2. 答案可能爆 `int`，用 `long long`。
3. 初始化可以直接 `add(i, a[i])`。

入门到进阶：P3368 树状数组 2

操作：

```
1 x y k: 区间 [x,y] 全部加 k
2 x: 查询 a[x]
```

直接树状数组不方便维护原数组，但可以维护差分数组：

```
d[i] = a[i] - a[i - 1]
a[x] = d[1] + d[2] + ... + d[x]
```

区间加：

```
add(l, k);
add(r + 1, -k);
```

单点查：

```
query(x);
```

这题的关键思想是：不是树状数组直接会区间加，而是差分把区间修改转成了两个单点修改。

进阶：P1972 HH 的项链

题意：多次询问区间 `[l,r]` 内不同颜色数量。

为什么想到树状数组？

单看一个询问，暴力统计要扫区间。多个询问如果每次清空数组会超时。

核心离线思想：

按右端点 r 排序询问。扫描数组位置 i ，维护每种颜色最后一次出现的位置。

当颜色 c 在位置 i 出现：

- 如果以前在 $last[c]$ 出现过，就在 $last[c]$ 位置加 -1 。
- 在当前 i 位置加 $+1$ 。
- 更新 $last[c] = i$ 。

这样树状数组中每种颜色只在“最后一次出现位置”贡献 1。

当扫到某个询问右端点 r 时，答案就是：

```
query(r) - query(l - 1)
```

坑点：

1. 询问必须按右端点排序。
2. 每种颜色只保留最后一次出现。
3. 如果颜色值很大，要用数组范围足够大或离散化。

四、ST 表：静态区间最值的 $O(1)$ 查询

1. 它的作用是什么？

ST 表适合：

- 数组不修改。
- 询问很多次。
- 问区间最大值、最小值、gcd。
- 要求查询很快，最好 $O(1)$ 。

2. 为什么会有这种结构？

如果数组静态，很多答案可以提前预处理。

ST 表提前存所有长度为 2^j 的区间答案。

定义：

$st[i][j]$ = 从 i 开始, 长度为 2^j 的区间答案

以最大值为例:

$st[i][j] = \max(a[i], \dots, a[i + 2^j - 1])$

3. 数学原理: 倍增 + 可重复贡献

长度为 2^j 的区间可以拆成两个长度为 $2^{(j-1)}$ 的区间:

$[i, i + 2^j - 1]$
=
 $[i, i + 2^{(j-1)} - 1]$
+
 $[i + 2^{(j-1)}, i + 2^j - 1]$

转移:

$st[i][j] = \max(st[i][j-1], st[i + (1 \ll (j-1))][j-1]);$

查询 $[l, r]$:

$len = r - l + 1$
 $k = \text{floor}(\log_2(len))$

用两个长度为 2^k 的块覆盖:

左块: $[l, l + 2^k - 1]$
右块: $[r - 2^k + 1, r]$

可能重叠, 但最大值、最小值、gcd 不怕重复:

$\max(x, x) = x$
 $\gcd(x, x) = x$

所以查询:

$\max(st[l][k], st[r - (1 \ll k) + 1][k])$

4. 为什么 ST 表不能普通求和？

因为两个块可能重叠。

最大值重复算没问题：

```
max(x, x)=x
```

但求和重复算会出错：

```
x+x=2x
```

所以 ST 表适合幂等运算：

- max。
- min。
- gcd。

不适合普通 sum。

5. 实现模板

```
int st[N][K];
int lg[N];

void build(int n) {
    for (int i = 2; i <= n; i++) {
        lg[i] = lg[i / 2] + 1;
    }
    for (int j = 1; j <= lg[n]; j++) {
        for (int i = 1; i + (1 << j) - 1 <= n; i++) {
            st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
        }
    }
}

int query(int l, int r) {
    int k = lg[r - l + 1];
    return max(st[l][k], st[r - (1 << k) + 1][k]);
}
```

6. Luogu 练习路线

入门：P3865 ST 表 & RMQ

问多次区间最大值，且要求查询 $O(1)$ 。

坑点：

1. `lg` 要预处理，不能每次 `log` 慢算。
2. `K` 要开够，例如 `1e5` 开 20，`1e6` 开 25。
3. 查询两个块可能重叠，但 `max` 不怕。

入门：P1816 忠诚

多次问区间最小值。和 P3865 本质一样，只是 `max` 换成 `min`。

进阶：P2880 Balanced Lineup

每次问区间最大值和最小值差。可以维护两个 ST 表：

```
mx[i][j]
mn[i][j]
```

答案：

```
queryMax(l,r) - queryMin(l,r)
```

进阶：P1890 gcd 区间

`gcd` 也满足重复贡献不改变答案，所以可以用 ST 表。

五、线段树：动态区间问题的大工具箱

1. 它的作用是什么？

线段树适合：

- 区间查询。
- 区间修改。
- 数据动态变化。
- 维护的信息可以由左右子区间合并。

比如：

- 区间和。
- 区间最大值。
- 区间最小值。
- 区间 gcd。
- 区间方差相关信息。
- 区间赋值 + 区间加 + 区间 max。

2. 为什么会有线段树?

树状数组很轻巧，但限制较多。它强在前缀和，适合可逆的前缀信息。

如果题目需要：

- 区间修改 + 区间查询。
- 维护 max/min/gcd。
- 多种修改叠加。
- 复杂区间信息。

树状数组会难写或不能写，线段树就出现了。

3. 数学原理：区间分治

线段树把 $[1, n]$ 递归二分：

```
[1,n]
├── [1,mid]
└── [mid+1,n]
```

每个节点维护自己区间的信息。

若维护区间和：

```
sum[p] = sum[p * 2] + sum[p * 2 + 1];
```

若维护最大值：

```
mx[p] = max(mx[p * 2], mx[p * 2 + 1]);
```

核心条件：大区间答案能由左右小区间合并。

4. 为什么查询是 $O(\log n)$?

查询 $[L, R]$ 时：

1. 当前区间完全被包含，直接返回当前节点。
2. 只和左儿子相交，递归左儿子。
3. 只和右儿子相交，递归右儿子。
4. 两边都相交，左右都查，再合并。

每层只访问少数节点，树高是 $O(\log n)$ 。

5. 为什么区间修改需要 lazy?

如果给 $[L, R]$ 每个数加 k ，暴力改所有叶子最坏 $O(n)$ 。

懒标记思想：

如果当前节点区间 $[l, r]$ 完全被修改区间覆盖，不继续往下改，只做：

```
sum[p] += (r - l + 1) * k;
lazy[p] += k;
```

意思是：

- 当前节点答案已经正确。
- 儿子还没有更新。
- 先记一笔账，等以后真的访问儿子时再下传。

这就是 lazy propagation。

6. 线段树区间加区间和模板

```
long long sum[N * 4], lazy[N * 4], a[N];

void push_up(int p) {
    sum[p] = sum[p * 2] + sum[p * 2 + 1];
}

void build(int p, int l, int r) {
    if (l == r) {
        sum[p] = a[l];
        return;
    }
    int mid = (l + r) / 2;
    build(p * 2, l, mid);
    build(p * 2 + 1, mid + 1, r);
    push_up(p);
}

void add_tag(int p, int l, int r, long long k) {
    sum[p] += (r - l + 1) * k;
    lazy[p] += k;
}

void push_down(int p, int l, int r) {
    if (lazy[p] == 0) return;
    int mid = (l + r) / 2;
    add_tag(p * 2, l, mid, lazy[p]);
    add_tag(p * 2 + 1, mid + 1, r, lazy[p]);
}
```

```

    lazy[p] = 0;
}

void update(int p, int l, int r, int L, int R, long long k) {
    if (L <= l && r <= R) {
        add_tag(p, l, r, k);
        return;
    }
    push_down(p, l, r);
    int mid = (l + r) / 2;
    if (L <= mid) update(p * 2, l, mid, L, R, k);
    if (R > mid) update(p * 2 + 1, mid + 1, r, L, R, k);
    push_up(p);
}

long long query(int p, int l, int r, int L, int R) {
    if (L <= l && r <= R) return sum[p];
    push_down(p, l, r);
    int mid = (l + r) / 2;
    long long ans = 0;
    if (L <= mid) ans += query(p * 2, l, mid, L, R);
    if (R > mid) ans += query(p * 2 + 1, mid + 1, r, L, R);
    return ans;
}

```

逐句理解重点：

- `N * 4`：线段树数组通常开 4 倍空间。
- `push_up`：孩子更新后，父亲重新合并。
- `add_tag`：当前区间整体加，不下钻。
- `push_down`：要访问孩子前，先把父亲欠孩子的标记传下去。
- `update`：完全覆盖就打标记，部分覆盖就递归。
- `query`：完全覆盖直接返回，部分覆盖递归查。

7. Luogu 练习路线

入门：P3372 线段树 1

操作：

区间加 k
区间求和

这是 lazy 线段树最基础模板。

坑点：

1. `sum` 和 `lazy` 要用 `long long`。
2. 递归访问儿子前必须 `push_down`。

3. 修改完儿子后必须 `push_up`。

进阶：P1253 扶苏的问题

操作：

区间赋值
区间加
区间最大值

难点：有两种 lazy：赋值和加法。

为什么难？

因为赋值会覆盖以前的加法。例如先把区间加 5，再赋值成 3，最后应该是 3，不是 8。

正确处理顺序：

- 赋值标记优先级高。
- 打赋值标记时，要清空加法标记。
- 加法遇到已有赋值标记时，可以直接把赋值值增加。

思维坑：

1. lazy 不是只有一种；要根据操作设计标记。
2. 多标记共存时，必须明确“后来操作覆盖前面操作”的关系。
3. `push_down` 时先传赋值，再传加法。

进阶：P1471 方差

操作：

区间加
查询区间平均数
查询区间方差

方差公式：

$$\text{Var} = E(x^2) - E(x)^2$$

所以节点维护两个信息：

sum1 = 区间和
sum2 = 区间平方和

当区间每个数加 `k`：

$$(x+k)^2 = x^2 + 2kx + k^2$$

于是：

```
sum2 += 2*k*sum1 + len*k*k
sum1 += len*k
```

为什么想到线段树？

因为区间加是动态修改，平均数和方差都能由区间和与平方和合并得到。

六、树状数组 vs 线段树：怎么选？

优先树状数组

当题目是：

- 单点修改 + 区间和。
- 区间修改 + 单点查询。
- 逆序对。
- 动态维护前缀可加信息。
- 信息比较简单。

树状数组代码短、常数小。

优先线段树

当题目是：

- 区间修改 + 区间查询。
- 维护 max/min/gcd/sum 等多种信息。
- 修改类型复杂，例如区间加、区间赋值同时存在。
- 查询信息复杂，例如方差。
- 后续可能接树链剖分、可持久化。

一句话：

```
树状数组像小刀：短、快、轻。
线段树像工具箱：重一点，但更通用。
```

七、LCA：树上路径问题的基础工具

1. 它的作用是什么？

LCA 是最近公共祖先。

在有根树里，两个点 u 和 v 的公共祖先中，深度最大的那个就是 LCA。

它常用于：

- 树上两点距离。
- 树上路径和。
- 树上差分。
- 树链剖分。
- 虚树。
- 最大生成树路径查询。

2. 为什么需要 LCA？

树上两个点的路径可以用 LCA 拆开：

$$u \rightarrow \text{lca}(u,v) \rightarrow v$$

距离公式：

$$\text{dist}(u,v) = \text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}(u,v)]$$

所以树上路径问题经常先变成 LCA 问题。

3. 数学原理：倍增

朴素做法：深的点一步一步往上跳，最坏 $O(n)$ 。

倍增定义：

$$\text{up}[x][j] = x \text{ 往上跳 } 2^j \text{ 步到达的祖先}$$

转移：

$$\begin{aligned} \text{up}[x][0] &= \text{parent}[x]; \\ \text{up}[x][j] &= \text{up}[\text{up}[x][j-1]][j-1]; \end{aligned}$$

意思是：跳 2^j 步等于先跳 $2^{(j-1)}$ 步，再跳 $2^{(j-1)}$ 步。

4. 查询过程

查询 `lca(u,v)` :

1. 保证 `u` 更深。
2. 把 `u` 跳到和 `v` 同一深度。
3. 如果 `u == v` , 答案就是它。
4. 从大到小枚举 `j` , 如果 `up[u][j] != up[v][j]` , 就让两者一起跳。
5. 最后 `u` 和 `v` 的父亲就是 LCA。

5. 实现模板

```
vector<int> g[N];
int depth[N];
int up[N][LOG + 1];

void dfs(int u, int father) {
    depth[u] = depth[father] + 1;
    up[u][0] = father;
    for (int j = 1; j <= LOG; j++) {
        up[u][j] = up[up[u][j - 1]][j - 1];
    }
    for (int v : g[u]) {
        if (v == father) continue;
        dfs(v, u);
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int j = LOG; j >= 0; j--) {
        if (diff & (1 << j)) u = up[u][j];
    }
    if (u == v) return u;
    for (int j = LOG; j >= 0; j--) {
        if (up[u][j] != up[v][j]) {
            u = up[u][j];
            v = up[v][j];
        }
    }
    return up[u][0];
}
```

逐句理解:

- `depth[u]` : `u` 的深度。
- `up[u][0]` : `u` 的父亲。
- `up[u][j]` : `u` 的 2^j 级祖先。

- 第一段 for：把深点拉平。
- 第二段 for：两点一起向上跳，但不能跳到同一个祖先，否则可能跳过 LCA。
- 最后返回父亲。

6. Luogu 练习路线

入门：P3379 最近公共祖先

题目直接问多次 LCA，是倍增模板。

坑点：

1. LOG 要开够。5e5 时 20 左右够，但实际可开 21 或 22。
2. 根的父亲设为 0，depth[0]=0。
3. DFS 可能爆栈，必要时用递归优化或改迭代。

进阶：P3398 仓鼠找 sugar

这类题往往要判断两条树上路径是否相交。可以借助 LCA 和祖先关系。

综合：P1967 货车运输

先用 Kruskal 建最大生成树，再用 LCA 查询路径最小边。

为什么用 LCA？

因为建出的最大生成树上，两点之间路径唯一，路径上的最小边就是答案；倍增表里可以同时维护祖先和路径最小边。

八、Trie：把字符串按字符路径组织起来

1. 它的作用是什么？

Trie 适合：

- 插入字符串。
- 查询字符串是否出现。
- 查询前缀是否存在。
- 统计某个前缀出现次数。
- 01-Trie 求最大异或。

2. 为什么需要 Trie？

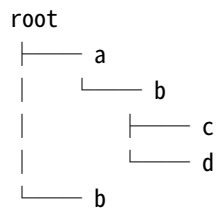
普通 `map<string, int>` 可以判断字符串是否存在，但 Trie 的优势在前缀。

它把字符串拆成字符路径。

例如插入：

abc
abd
b

Trie:



每个节点代表一个前缀。

3. 怎么实现?

```
int trie[N][26];
int cnt_node;
int state[N];

void insert_string(const string& s) {
    int p = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[p][c] == 0) trie[p][c] = ++cnt_node;
        p = trie[p][c];
    }
    state[p] = 1;
}

int query_string(const string& s) {
    int p = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[p][c] == 0) return 0;
        p = trie[p][c];
    }
    if (state[p] == 0) return 0;
    if (state[p] == 1) {
        state[p] = 2;
        return 1;
    }
    return 2;
}
```

逐句理解：

- `trie[p][c]`：从节点 `p` 走字符 `c` 到哪个节点。
- 根节点编号为 0。
- `cnt_node`：当前创建了多少节点。
- `state[p] = 1`：该节点是完整字符串结尾，且还没被查询过。
- `state[p] = 2`：已经查询过。

4. 做题时怎么想到？

看到这些信号：

- 很多字符串。
- 只包含小写字母或 01 字符。
- 查询前缀。
- 查询字符串集合。
- 要统计前缀数量。
- 异或最大值。

就想 Trie。

5. Luogu 练习路线

入门：P2580 于是他错误的点名开始了

查询名字：

- 不存在：`WRONG`。
- 存在且第一次出现：`OK`。
- 存在但重复出现：`REPEAT`。

坑点：

1. 路径存在不等于字符串存在。例如插入 `abcd`，查询 `abc`，路径有但不是完整名字。
2. 查询后要改变状态，防止重复点名。
3. 节点总数大约是字符串总长度，不是字符串数量。

入门到进阶：P8306 字典树模板

训练前缀计数。

节点可以维护：

`cnt[p]` = 有多少字符串经过这个节点

查询前缀时走到对应节点，输出 `cnt[p]`。

进阶：P4551 最长异或路径

先在树上 DFS 求每个点到根的路径异或值 `dis[x]`。

任意两点路径异或：

$\text{dis}[u] \text{ xor } \text{dis}[v]$

于是问题变成：给一堆数，找两个数异或最大。

用 01-Trie，每一位尽量走相反位。

为什么想到 Trie？

因为异或最大值是按二进制位贪心，而 01-Trie 正好按位组织数字。

九、平衡树 / 权值线段树：动态维护有序集合

1. 它的作用是什么？

平衡树维护动态有序集合，支持：

- 插入。
- 删除。
- 查询排名。
- 查询第 k 小。
- 查询前驱。
- 查询后继。

2. 为什么需要平衡树？

`set` 可以插入、删除、前驱、后继，但不能直接查排名和第 k 小。

如果用 `vector`：

- 插入删除要移动元素， $O(n)$ 。
- 查询第 k 小虽然容易，但动态维护很慢。

平衡树希望所有操作都是 $O(\log n)$ 。

3. 数学原理：二叉搜索树 + 子树大小

二叉搜索树满足：

左子树所有值 < 当前节点值 < 右子树所有值

如果每个节点维护：

$\text{size}[\text{node}] = \text{子树中元素个数}$
 $\text{cnt}[\text{node}] = \text{当前值出现次数}$

就能查询排名和第 k 小。

查询第 k 小：

- 如果 $k \leq \text{size}[\text{left}]$ ，去左子树。
- 如果 $\text{size}[\text{left}] < k \leq \text{size}[\text{left}] + \text{cnt}[\text{node}]$ ，答案是当前节点。
- 否则去右子树，且 $k -= \text{size}[\text{left}] + \text{cnt}[\text{node}]$ 。

为什么要平衡？

普通 BST 插入 $1, 2, 3, 4, 5$ 会退化成链，操作变 $O(n)$ 。

平衡树通过旋转或随机优先级让高度维持在 $O(\log n)$ 。

常见实现：

- Treap。
- FHQ Treap。
- Splay。
- 权值线段树。
- PBDS。

初学建议：先理解权值线段树和 Treap 思想。

4. 权值线段树替代平衡树

如果值域不大，或者可以离散化，可以用权值线段树。

普通线段树的下标是数组位置。

权值线段树的下标是值域。

例如值域 $[1, 100000]$ ：

- 叶子 x 表示数字 x 出现几次。
- 节点维护这个值域区间内数字总数。

查询第 k 小：

- 如果左子树数量 $\geq k$ ，去左边。
- 否则去右边， $k -= \text{左子树数量}$ 。

这和平衡树本质一样，都是用 size 引导搜索方向。

5. Luogu 练习路线

入门：P3369 普通平衡树

操作：

- 1 插入 x
- 2 删除 x
- 3 查询 x 的排名
- 4 查询排名为 x 的数
- 5 查询 x 的前驱
- 6 查询 x 的后继

为什么想到平衡树？

因为题目关键词是动态有序集合 + 排名 + 第 k 小 + 前驱后继。

坑点：

1. 是可重集合，重复值要用 `cnt`。
2. 删除时只删一个，不是删全部。
3. 查询前驱后继时，题目保证答案存在，但不保证 x 本身存在。

进阶：P6136 普通平衡树加强版

它是 P3369 加强版，强制在线，数据更大。

强制在线的意思是：每次真实输入和上一次答案有关，你不能把操作全部读完后离线处理。

所以必须动态维护结构。

替代练习：权值线段树版 P3369

如果值域可以离散化，你可以用权值线段树实现六种操作。这样能帮助你从“线段树维护数组区间”升级到“线段树维护值域区间”。

十、可持久化线段树：维护历史版本

1. 它的作用是什么？

可持久化线段树解决：

- 查询历史版本。
- 在旧版本上修改。
- 每次操作生成新版本。
- 主席树区间第 k 小。

2. 为什么需要它？

如果每次生成新版本都复制整个数组：

每次 $O(n)$
总共 $O(nm)$

太慢。

普通线段树单点修改只影响从根到叶子的一条路径。

其他节点完全不变，可以共享。

所以可持久化线段树每次只新建 $O(\log n)$ 个节点。

3. 数学原理：结构共享

每个版本保存一个根：

root[0]
root[1]
root[2]

修改版本 v 时：

- 旧节点不改。
- 经过的节点复制一份。
- 没经过的子树继续指向旧版本节点。

这样旧版本不会被破坏，新版本也能复用旧结构。

4. 可持久化数组模板思路

```
struct Node {
    int left, right;
    int val;
} tree[N * 25];

int root[N], tot;

int update(int old, int l, int r, int pos, int val) {
    int now = ++tot;
    tree[now] = tree[old];
    if (l == r) {
        tree[now].val = val;
        return now;
    }
}
```

```

int mid = (l + r) / 2;
if (pos <= mid) {
    tree[now].left = update(tree[old].left, l, mid, pos, val);
} else {
    tree[now].right = update(tree[old].right, mid + 1, r, pos, val);
}
return now;
}

int query(int p, int l, int r, int pos) {
    if (l == r) return tree[p].val;
    int mid = (l + r) / 2;
    if (pos <= mid) return query(tree[p].left, l, mid, pos);
    return query(tree[p].right, mid + 1, r, pos);
}

```

最关键的一句：

```
tree[now] = tree[old];
```

先复制旧节点，再只改其中一个儿子。

5. Luogu 练习路线

入门：P3919 可持久化数组

操作：

```

v 1 p c: 基于版本 v，把 a[p] 改成 c，生成新版本
v 2 p:  查询版本 v 的 a[p]，并生成一个相同的新版本

```

坑点：

1. 查询操作也会生成新版本。
2. 修改不能改旧节点，一定要复制。
3. 节点数组要开足，通常是 $n + m \log n$ 级别。

进阶：主席树求静态区间第 k 小

核心思想：

```
root[i] = 前 i 个数构成的权值线段树
```

那么区间 $[l, r]$ 的信息等于：

```
root[r] - root[l - 1]
```

查第 k 小时，在两棵树上同时走。

左边数量：

```
cntLeft = sum[left(root[r])] - sum[left(root[l - 1])]
```

如果 `cntLeft >= k`，答案在左子树；否则去右子树，`k -= cntLeft`。

这其实是：

前缀和思想 + 权值线段树 + 可持久化

十一、文档中提到但未详细展开的结构地图

这些结构在你的文档开头或结尾出现过，但不是当前第一优先级。你可以先知道“看到什么题想它们”。

1. 前缀和

适合：静态区间和。

核心：

```
s[i] = s[i - 1] + a[i];  
range_sum(l, r) = s[r] - s[l - 1];
```

看到：数组不修改、多次区间和，先想前缀和。

2. 差分

适合：区间修改，最后统一求值；或配合树状数组做区间加单点查。

核心：

```
d[l] += k;  
d[r + 1] -= k;
```

看到：很多次区间加，最后输出数组，想差分。

3. 单调栈 / 单调队列

单调栈：找每个位置左/右第一个比它大或小的元素。

单调队列：滑动窗口最大值/最小值。

看到：最近更大、更小、滑动窗口最值，想它们。

4. 堆

适合：动态维护最大/最小元素。

看到：每次取当前最小、最大，或者 Dijkstra 堆优化，想优先队列。

5. 拓扑排序

适合：有向无环图上的先后依赖关系。

看到：课程先修、任务依赖、是否有环、DAG DP，想拓扑排序。

6. KMP

适合：单模式串匹配。

看到：在文本串中找模式串出现位置，想 KMP。

核心是 `next` 数组，维护模式串前后缀匹配信息。

7. 字符串哈希

适合：快速比较两个子串是否相等。

看到：大量子串比较、判重，想哈希。

8. AC 自动机

适合：多模式串匹配。

它是 Trie + fail 指针。

看到：很多关键词同时在文本中匹配，想 AC 自动机。

9. 树链剖分

适合：树上路径修改、路径查询、子树修改、子树查询。

它把树上路径拆成若干段连续 DFS 序区间，再交给线段树维护。

Luogu 练习：P3384 重链剖分 / 树链剖分。

10. 分块

适合：操作复杂但可以用根号复杂度折中。

典型思想：把数组分成大约 $\sqrt{n}$ 个块，整块快速处理，散块暴力处理。

Luogu 练习：P2801 教主的魔法。

11. 莫队

适合：静态数组，多次离线区间询问，且可以用 `add/remove` 维护答案。

核心：把询问排序，让左右指针少移动。

Luogu 练习：P2709 小 B 的询问。

12. 可持久化并查集

适合：并查集也需要历史版本。

它通常用可持久化数组维护 `fa` 和 `rank/dep`。

这属于进阶内容，建议在线段树和可持久化线段树熟练后再学。

十二、总题单：从浅到深练习

邻接表

1. P5318 查找文献：DFS/BFS + 邻接表排序。
2. P3379 最近公共祖先：邻接表存树 + LCA。
3. P4551 最长异或路径：带权树邻接表 + 01-Trie。

并查集

1. P3367 并查集：模板。
2. P1197 星球大战：离线反向加点。
3. P1967 货车运输：Kruskal + 并查集 + LCA。

树状数组

1. P3374 树状数组 1：单点修改 + 区间和。
2. P3368 树状数组 2：差分 + 树状数组。
3. P1972 HH 的项链：离线 + 树状数组。

ST 表

1. P3865 ST 表 & RMQ：区间最大值。
2. P1816 忠诚：区间最小值。
3. P2880 Balanced Lineup：区间最大最小差。
4. P1890 gcd 区间：区间 gcd。

线段树

1. P3372 线段树 1：区间加 + 区间和。
2. P1253 扶苏的问题：区间赋值 + 区间加 + 区间最大值。
3. P1471 方差：维护与平方和。

LCA

1. P3379 LCA 模板。
2. P3398 仓鼠找 sugar：路径关系判断。
3. P1967 货车运输：最大生成树上倍增查询。

Trie

1. P2580 错误的点名：字符串存在性 + 状态。
2. P8306 字典树模板：前缀计数。
3. P4551 最长异或路径：01-Trie。

平衡树 / 权值线段树

1. P3369 普通平衡树。
2. P6136 普通平衡树加强版。
3. 用权值线段树重做 P3369。

可持久化

1. P3919 可持久化数组。
2. 静态区间第 k 小：主席树模板题。

后续扩展

1. P3384 树链剖分。
2. P2801 分块。
3. P2709 莫队。

十三、最终做题判断口诀

看到“图很大、边数 m、需要遍历”：邻接表。

看到“合并、连通、是否同集合”：并查集。

看到“单点修改 + 区间和”：树状数组。

看到“区间修改 + 单点查询”：差分 + 树状数组。

看到“静态区间最大/最小/gcd，多次询问”：ST 表。

看到“区间修改 + 区间查询”：线段树 + lazy。

看到“树上两点路径、祖先、距离”：LCA。

看到“字符串集合、前缀、字典”：Trie。

看到“动态第 k 小、排名、前驱后继”：平衡树 / 权值线段树。

看到“历史版本”：可持久化线段树 / 主席树。

看到“树上路径修改查询”：树链剖分。

看到“静态离线区间询问，可以左右指针移动维护”：莫队。

看到“操作复杂但可以根号折中”：分块。

最重要的一句话：

数据结构不是模板名，而是信息组织方式。
题目反复做什么操作，你就设计什么结构去维护它。